Hong Kong Metropolitan University

COMP2090SEF
Data Structures, Algorithms And Problem Solving
Course Project Report
Group 33
Submission date:

We declare that:
(i) all members of the group have read and checked that all parts of the project (including proposal, code programs, and reports), irrespective of whether they are contributed by individual member or all members as a group, here submitted is original except for source material explicitly acknowledged;
(ii) the project, in parts or in whole, has not been submitted for more than one purpose without declaration;
(iii) we are aware of the University's policy and regulations on honesty in academic work and understand the possible consequence when breaching such policy and regulations;
(iv) we confirm that we have declared in the report about the usage of AI and other generative models, including but not limited to ChatGPT, LaMA, Gemini, Mistral, and Stable Diffusion, and complied with the instructions provided by HKMU;
(v) we are aware that all members of the group should be held responsible and liable to disciplinary actions, irrespective of whether he/she has signed the declaration and whether he/she has contributed, directly or indirectly, to the problematic contents. We confirm that we have read through and understood the project requirements. We understand that failure to comply with the project requirements will result in score deduction.

NAME	SID
fangtianxing	13979884
zhouqixin	14010563
zhaowenzong	14132007

2. The group must acknowledge ALL external resources or code from others referenced. We will only mark the parts that are original, not the parts from others.
3. All members are liable to the plagiarism incidence, even if a free-rider.

Course Project Report: Library User Registration System

1. Core Functions and Technical Summary

This project is a simple library user registration system built with Python, which verifies user information step by step to ensure only valid data is used for registration. The execution logic of the system is clearly visualized in Appendix A: System Flow Diagram, and the OOP design of the project is presented in Appendix B: Class Structure Diagram.

1.1 Main Functions

- Name Check(*enroll_library*): Prompts users to enter a name, with three rules: the name cannot be empty, the first character cannot be a number, and the rest must be lowercase letters. Invalid inputs trigger a re-prompt until a valid name is provided.
- Gender Check(*_sex*): Only accepts male or female as input. Any other entry will be rejected, and users need to re-enter. Valid gender input will automatically start the year check process.
- Year C h*eck*(*_year*) : Requires users to input a number between 1 and 4. It uses try-except to catch non-numeric inputs; if the number is out of range, users will be asked to re-enter. After passing the check, self.year will be saved.
- Registration Completion (*complete_registration*) : Checks whether name ,sex , and year are all filled and valid. If yes, it prints the successful registration information; if any item is missing or invalid, it prompts the registration failure.

1.2 Technical Summary

- The entire system is encapsulated in the Library class, and the user's information and verification logic are integrated in the class instance.
- Use while True loop to realize continuous input verification until the user enters valid information.
- Use string methods isdigit() and islower() to complete the format check of the name.
- Use try-except ValueError exception handling mechanism to avoid program crash caused by non-numeric input in the year check.
- The system runs according to the fixed process: name check → gender check → year check → registration completion, which is fully demonstrated in the system flow diagram.

2. Application of OOP Concepts

This project fully applies the core ideas of object-oriented programming, and the specific manifestations are as follows:

- Encapsulation: The user's data attributes (self.name , self.sex , self.year) and functional methods (various verification functions) are all encapsulated in the Library class. External code only needs to create class instances and call methods to complete registration, without caring about the internal implementation logic of the methods. All attributes and methods of the Library class are clearly listed in the class structure diagram.
- Instance Methods: All verification functions are designed as instance methods of the class, and the self parameter is used to realize the access and modification of the instance attributes. Each instance can independently complete a user's registration process without interfering with each other.
- Single Responsibility Principle: Each method is only responsible for one type of verification, which makes the code structure clear, easy to read and debug. For example, _sex_() only handles gender verification, and _year_() only handles year verification.

3. Declaration of External Resources

This project is independently developed by the group members, and only uses the built-in functions and standard libraries of Python 3. No third-party software packages, open-source code segments or artificial intelligence generation tools are used in the development process. All codes and logical designs comply with academic integrity requirements.

4. Self-Reflection and Future Improvements

4.1 Existing Weaknesses

- The gender verification function is too rigid, only accepting lowercase male and female , and cannot recognize input formats such as uppercase or abbreviations (e.g., Male , M).
- Error prompt information is too simple, only prompting "invalid input" but not explaining the specific error reasons and correction methods.
- The system lacks data persistence function, and the user's registration information is only printed on the console and cannot be saved for a long time.
- The name verification logic is not strict enough, allowing special symbols and numbers in non-initial positions, which is inconsistent with the actual name usage habits.

4.2 Future Improvement Directions

- Optimize the gender verification logic, add the function of converting input to lowercase uniformly, and support abbreviated input such as M and F to improve user experience.
- Refine the error prompt information, clearly point out the specific problems of the input (e.g., "the first character of the name cannot be a number"), so that users can correct the input quickly.
- Add data persistence function, and save the registered user information to files such as TXT or CSV, so that the data can still be retained after the program is closed.
- Strengthen the name verification logic, restrict the input to only letters and spaces, and make the system more in line with the actual application scenarios.

5. GitHub Repository Link

[Put your GitHub repository link here:https://github.com/ngn8255-bot/Aa121212/tree/master]
(The repository contains the complete code of this project and all supporting files including the two diagrams in the appendix.)

6.Reference

This project is the original work of the author(s). No third-party libraries, open-source code snippets, or generative artificial intelligence tools were used in the development process. All logical designs and code implementations comply with academic integrity requirements.

Appendix A：

```
borrow
C:\Users\28954\PycharmProjects\PythonProject\.venv\Scripts\python.ex
enter your name: fang
name is valid, registration can proceed
enter your sex (male/female): male
sex is valid
enter year (1-4): 3
year is valid
registration completed!
name: fang, sex: male, year: 3.0

进程已结束，退出代码为 0
```

```
C:\Users\28954\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\28954\PycharmProjects\PythonProject\borrow.py
enter your name: 122A
invalid name! first character cannot be a number, and rest must be lowercase
enter your name: we
name is valid, registration can proceed
enter your sex (male/female): male
sex is valid
enter year (1-4): 3
year is valid
registration completed!
name: we, sex: male, year: 3.0

进程已结束，退出代码为 0
```

This is my screenshot after my code running

Appendix B:
Class Name: Library
- Attributes:
    1. `name`: A string to store the user's valid name.
    2. `sex`: A string to store the user's valid gender (`male` or `female`).
    3. `year`: A float to store the user's valid academic year (between 1 and 4).
- Methods:
    1. `enroll_library(self)`: Validates the user's name format and stores it.

2. `_sex_(self)`: Validates the user's gender input and stores it.

3. `_year_(self)`: Validates the user's academic year input (with error handling) and stores it.

4. `complete_registration(self)`: Verifies all information is complete and outputs the registration result.